# FDE India - Take Home Exercise

## Background (10 minutes)

### About myself

I'm Kartik Garg. I studied Computer Science at IIIT Hyderabad and started my full-time career at Sprinklr, where I worked on CRM data imports, job scheduling, and the announcements framework. Since June 2025, I've been at Quizizz on backend and platform work across search and the Resource Library. Across both roles, I've gravitated toward problems that combine scale, ambiguous requirements, and the need to deliver safely. At Quizizz, I led a zero-downtime migration of more than 400 million documents from Elasticsearch to PostgreSQL. At Sprinklr, I helped cut the projected duration of a CRM import from 72 days to 8, against a 15-day business deadline.

### Why this role

I enjoy the point where customer context meets engineering reality: asking questions until the underlying need is clear, mapping it to the system's constraints, and taking responsibility through implementation, validation, and handoff. The FDE role at Notion is a natural fit for the migration, integration, and platform work I have done, and for the customer-facing engineering work I want to do more of.

### Career arc and what I bring

- Sprinklr (July 2023–June 2025): integration-heavy platform work - CRM imports, scheduling, asynchronous jobs, and customer-facing reliability. The work taught me to surface delivery risk early and make systems easier to extend.
- Quizizz (June 2025–present): broader ownership across search and the AI-powered Resource Library. I have worked from teacher and product feedback through platform design, scale, and production rollout.
- My working style:

    - I work backwards from the user outcome.
    - Once I understand the requirement, I clarify the system and delivery constraints.
    - That makes trade-offs visible early.
    - I discuss options with peers, consider their input, and align on a design that works for the team.
    - I leave behind a system the next team can operate.
- More Details of Work Experience
    - Resource Library: teachers repeatedly searched for the same material each year. We organized resources around the structure teachers already use - standards and books, with chapters, sections, and subsections - so they could find relevant material with less friction.
    - Platform outcomes:
        - The Resource Library re-architecture reduced infrastructure cost by 85% (about $20 to $3 per run), database load by 70%, and code duplication by 60%
        - The content pipeline reduced curation time from 96 hours to 2 hours.
    - Business Outcomes: $80000 in revenue this year, expected to double next year with incoming high school initiatives.
    - A useful reflection:
        - I learned to avoid generalizing too early.
        - I initially proposed a strategy/factory abstraction while requirements were still changing.
        - But eventually we started simpler and introduced a reusable abstraction once the product shape was clearer.

# Hiring Manager + Technical Delivery (20 minutes)

# Technical engagement: Elasticsearch to PostgreSQL migration

The Organization started an initiative to cut down infra costs, and i was asked to do an analysis of our search engine on My Library. My Library is a place which every user visits at least once in every session. This is the place where they find their previously hosted resources, created ones, shared ones.

After Analyzing last 10 years of search workload (pushed to Bigquery as analytics), i figured that  the users are primarily searching direct-name with limited fuzziness. A proof of concept showed that PostgreSQL could support the needed experience, with a simpler and less expensive operating footprint than Elasticsearch. The engagement was to move more than 400 million documents without interrupting the search experience or losing changes made while the backfill was running.

## Scope and requirements

- Maintain uninterrupted customer-facing search throughout the migration.
- Keep Elasticsearch available as a rollback path until the PostgreSQL read path met the correctness and performance bar.
- Account for creates, updates, and deletes during the 8–10 hour backfill. The Elasticsearch scroll represented a snapshot, so later writes needed to be captured and replayed.
- Prove data parity and query performance before changing the primary read path
    - We tracked success using product and business measures such as:
        - What level of data drift did we observe?
        - How did API latency change, and what impact did it have on user wait time?
        - How did changes in top search results affect the overall host rate?
- Create the required query-driven indexes for the access patterns established early in the proof of concept and from the production Elasticsearch load as well.

- Choose the partitioning strategy based on query patterns and measured performance
    - We finally created 64 partitions by user ID. Since production queries filtered on user ID, partition pruning could reduce the search space by up to 64×.

## What I personally owned and how delivery worked

I led the migration design and rollout. I designed the partitioned data model and the safety mechanisms around the backfill: dual writes, a change-event queue to capture concurrent mutations, replay after the initial load, reconciliation, shadow reads, and staged traffic changes. I treated the rollout as a sequence of evidence-based gates rather than a one-time cutover.

- Elasticsearch remained primary while PostgreSQL was populated as the secondary destination.
- We replayed queued changes, reconciled the datasets, and shadow-read/compared results to measure drift and validate query behavior.
- After the checks passed, PostgreSQL became primary while Elasticsearch remained available as fallback; only after confidence was established did we retire the old path.

I aligned the rollout around the concerns each partner needed answered: product confidence in an uninterrupted search experience, engineering confidence in correctness and performance, and operational confidence in detection and rollback. In stakeholder conversations, I made those criteria and the phase gates explicit so that the decision to advance was based on observed results.

## Cross-functional and user impact

The work connected a backend storage and search decision to a user-facing experience. I kept search availability, result quality, query behavior, data correctness, and operating cost in the same decision frame, and aligned product and engineering partners around those outcomes.

## Impact

- Migrated 400M+ documents with zero customer impact and less than 0.15% consistency drift.
- Improved query p99 latency by about 25%.
- Reduced infrastructure cost by approximately $20,000 per year.

## A particularly challenging delivery: Sprinklr CRM import

At Sprinklr, a customer needed roughly 180 million CRM records imported within 15 days. A staging proof of concept on 15 million records projected about 72 days for the full import. I raised the schedule risk before production rollout, investigated the throughput, and found that the importer made individual database calls record by record.

- I optimized the database interaction by batching work, then proposed reorganizing the monolithic importer around shared operations and distinct strategies for insert, update, delete, upsert, and sync.
- Because the code path served multiple customers, I built an internal engineering test suite around the shared importer operations.
- The projected full import fell from 72 days to 8 days, meeting the 15-day deadline with roughly a week of buffer. The clearer separation of shared operations and import strategies also reduced new-developer onboarding time by about 90%.

What made it difficult was the combination of a hard customer deadline, a large dataset, and a design whose per-record database calls could not meet the required throughput. The turning point was measuring the staging run, making the risk visible early, and addressing both the immediate bottleneck and the maintainability issue without skipping validation.

**Stakeholder involvement**

- QA validated the importer in staging and expanded product-level coverage before production rollout.
- Product partners helped define an ‘unchanged’ import status, avoiding unnecessary database updates.

- I presented the re-architecture proposal and rollout plan to engineering leadership.

# Technical Integration Scenario (30 minutes)

I built a custom, one-way Plane → Notion connector. It gives people who plan and communicate in Notion a reliable view of active Plane work, while Plane remains the source of truth for task state.

Github Link: https://github.com/krtikgrg/plane-notion-connector

Architecture Diagram: https://github.com/krtikgrg/plane-notion-connector/blob/main/docs/architecture.pdf

## The problem and the value

The use case I designed for is a team that executes work in Plane but shares plans, decisions, and status in Notion. Project leads and cross-functional partners who work primarily in Notion need current delivery context without repeatedly switching tools or maintaining a second manual status list. Manual copies drift; two-way writes introduce ownership and conflict-resolution problems.

- The connector publishes a read-only projection of selected Plane work into a Notion workspace. Each item retains a direct link to Plane, where the authoritative record can be changed.
- The value is less context switching, with project work visible beside the documents and decisions that explain it. The prototype demonstrates the workflow end to end
- I chose Plane because
    - In some sense plane is a competitor of Notion
    - Notion Marketplace has integrations available for JIRA and Asana
    - it makes a useful integration problem:
        - resolve workspace/project/task context
        - map work-item data
        - maintain stable identity, and present the result in Notion.

- This is a custom API integration rather than a marketplace connector.

## What I built and where I drew the line

The prototype is a Java Spring Boot service with a React + TypeScript / Vite control UI and MongoDB. The same Spring Boot service serves the UI and REST API and runs one in-process background worker.

- Configure a Plane project, choose work-item sync options and a schedule, and see provisioning or update status in the UI.
- Run a full project sync, or paste a Plane task URL for a one-item sync. A matching project config selects its own Notion destination
- Edge Case: An unconfigured task URL goes to one shared, manual-only destination.
- The prototype syncs active, non-draft work items from Plane to Notion. Notion is read-only with respect to Plane.
- Comments, attachments, broad history migration, and two-way write-back are outside this implementation.

### How I prioritized scope

I prioritized for a vertically complete, trustworthy outcome

1. Prove the vertical slice: resolve a Plane project or task, map stable identity and useful status, create or update the matching Notion page, and retain the link to the source. If this loop did not work end to end, scheduling and UI polish would not prove the integration.
2. Make repeat use trustworthy: stable mappings and hashes prevent duplicate pages and skip unchanged writes; checkpoints keep scheduled syncs incremental; run history and visible operation states tell the user whether the work is pending, complete, or failed.
3. Make it useful in the team’s Notion workflow: organize project data under workspace/project pages, support a configured project sync, and provide a shared destination for a one-off task URL when there is no project config.

I deferred features when they would expand the contract or require guarantees the prototype did not yet provide:

- Two-way writes: Plane remains authoritative; editing both systems requires conflict resolution and a clear ownership policy.

## Key engineering decisions

- **Clear source of truth:** Keep Plane authoritative with one-way sync into Notion, avoiding bidirectional conflicts.
- **Single deployable application:** Run the API and background worker together, with separate controller, service, repository, and integration layers.
- **Shared durable job queue:** Use MongoDB for configuration operations, manual syncs, and scheduled runs, with atomic job claims and persistent history.
- **Typed integration boundaries:** Use dedicated provider clients and explicit payload models, with shared HTTP transport, timeouts, and error handling.
- **Repeatable, recoverable writes:** Combine source mappings, content hashes, and pending-create records to skip unchanged writes and recover from uncertain creation outcomes.
- **Bounded incremental processing:** Use paginated reads, successful-run checkpoints, overlapping time windows, and batched persistence to limit processing and preserve progress.
- **Automatic Notion provisioning:** Create the page hierarchy and typed data-source schema, including evolving state and label options.
- **Private-network compatibility:** Run alongside self-hosted Plane and connect outbound to Notion, with configurable endpoints and server-side credentials.
- Full archive/delete/move reconciliation: archive and unarchive can leave Plane updated_at unchanged, so checkpoint polling alone cannot promise lifecycle completeness. A full reconciliation or reliable event source should come first.
- Multi-worker recovery and timezone/DST handling: the demo uses one worker and UTC scheduling; these are production-hardening requirements once deployment scale and customer schedules are known.

## End-to-end architecture

The browser sends config and sync requests to the Spring REST API. The API persists work to MongoDB's durable sync_jobs queue and returns promptly. A single worker atomically claims due jobs and dispatches manual/scheduled syncs or configuration operations.

### Why configuration creation runs in the worker

Configuration creation has more work than validating a form: it provisions Notion pages and a data source, refreshes hierarchy summaries, and seeds the schedule. Keeping those remote calls off the request path avoids making save latency depend on Notion. The API saves a PROVISIONING config, queues a durable CONFIGURATION / CREATE job, and returns HTTP 202. The UI polls until the worker reports READY or FAILED. Config updates keep the previous effective settings in force and hold queued syncs until the worker applies the change. A local smoke returned HTTP 202 in 70 ms; Notion provisioning completed asynchronously and the config reached READY. This is one development observation, not a production latency target.

Trimmed excerpts from ConfigService and SyncJobWorker:

```
SyncConfigDocument saved = configRepository.save(draft);
try {
    jobRepository.save(newOperationJob(ConfigOperationType.CR
EATE, saved, null));
} catch (RuntimeException exception) {
    configRepository.deleteById(configId);
    throw exception;
}
return saved;
```

```
@Scheduled(fixedDelayString = "${sync.worker.poll-interval-m
s:5000}")
public void claimAndRunOne() {
    jobRepository.claimNextDueJob(Instant.now()).ifPresent(th
is::dispatch);
```

```
}

private void dispatch(JobDocument job) {
    if (job instanceof ConfigOperationJobDocument operation)
{
        runConfigurationOperation(operation);
        return;
    }
    // Manual and scheduled sync jobs go to the sync handler.
}
```

- For a sync, PlaneClient resolves the submitted workspace, project, or work-item URL and reads Plane data through the Plane API. Project syncs page through active, non-draft work items and use an updated_at checkpoint.

## Prototype evolution: full scans to timestamp-bounded syncs

In the first cut, every run of a project sync config scanned the entire eligible Plane project; reads had no updated_at window, so unchanged work was revisited on each run. On reviewing the code from a production lens, this seemed to be a problem because organizations can have large number of tasks.

Over the next iterations, I added timestamp-based incremental reads to avoid that repeat work. In the current design, a new config still gets one full backfill. After a successful run, each repeat sync reads active, non-draft items from the previous successful project checkpoint (with a 100 ms overlap) through a cutoff fixed at the current run's start. The overlap replays timestamp ties, later changes are picked up in the next window, and a failed run does not advance the checkpoint. Plane returns the filtered results in 200-item cursor pages.

Trimmed excerpt from SyncJobWorker:

```
Instant upperCutoff = job.startedAt() == null ? Instant.now()
: job.startedAt();
Instant checkpoint = jobRepository.findLatestSuccessfulProjec
tSyncStart(config.id())
        .map(startedAt -> startedAt.minus(CHECKPOINT_OVERLA
P)).orElse(null);
```

```
return workItemSyncService.syncProject(project, checkpoint, u
pperCutoff, progress::set);
```

## Bulk sync: bounded batches and measured evidence

Latency work focused on two parts I could improve locally: timestamp checkpoints skip unchanged Plane work, and bounded mapping batches reduce MongoDB round trips. Plane returns 200-item pages; WorkItemSyncService buffers up to 1,000 work items, fetches mappings with one data-source-scoped `$in` query per batch, then bulk-upserts mappings. While searching for a Notion bulk write API, I found the documented Create a page and Update a page operations are page-scoped; I did not find a multi-page write endpoint. So these batches reduce source and Mongo work, while each changed Notion page still needs its own request. Notion's request limits vary by workspace plan.

Trimmed excerpt from WorkItemSyncService:

```
private static final int MONGO_BATCH_SIZE = 1000;

List<WorkItem> batch = new ArrayList<>(MONGO_BATCH_SIZE);
planeClient.forEachProjectWorkItemPage(project.planeWorkspace
Slug(), project.planeProjectId(), updatedAtFrom,
        updatedAtTo, page -> {
            for (WorkItem workItem : page) {
                batch.add(workItem);
                if (batch.size() == MONGO_BATCH_SIZE)
                    processBatch(notionDataSourceId, project,
batch, accumulator, progressConsumer);
            }
        });
if (!batch.isEmpty())
    processBatch(notionDataSourceId, project, batch, accumula
tor, progressConsumer);
```

Earlier smoke runs recorded a 50-item seed in 24.2 seconds and a 2-item incremental create/update in 2.9 seconds. After the hierarchy changes, I reran 15

synthetic projects across three workspaces: all 15 syncs succeeded, creating 771 Notion pages for 771 eligible work items, with 0 updates, unchanged items, or failures. From first enqueue to last completion took 9m40s- about 80 page creates per minute (1.33 per second) end to end. Config provisioning is excluded; this measures Plane reads, Mongo mapping work, individual Notion writes, and hierarchy-summary refreshes. It is the prototype's observed performance mark, not an SLA or a measurement of Notion's API ceiling.

- The worker routes each item to the configured project destination, or to the shared ad hoc destination for an unconfigured task URL.
- NotionClient writes project databases/data sources and work-item pages. NotionBlockClient handles block reads/writes, hierarchy summaries, and page archival.
- MongoDB stores project metadata, configuration and schedule state, durable jobs/run history, the source-to-Notion mapping and source hash, and destination IDs. The UI shows operation states and sync results from the service.

For the current demo, the service runs locally and calls Plane and Notion APIs. If deployed beside a private Plane instance, the poller can run inside a network that can reach Plane and make outbound calls to Notion; it does not require a public inbound webhook.

## How the integration uses Notion

The Notion hierarchy is a parent page, then workspace pages, then a project page with an inline work-items database. In Notion's model, each database row is itself a page; the data source defines the properties and the table view presents those pages. The Notion connection is granted access to the parent and destination pages; its token stays in backend environment configuration and is not stored in MongoDB.

- A work-item page includes the Plane title and identifier, stable Plane item ID, direct Plane URL, description, exact state name (including custom states), priority, assignees, labels, due date, and source update time.
- The configured project database contains one project's work. The shared ad hoc database can contain work from multiple projects, so it also stores

workspace and project identity.

- Workspace and parent summaries roll up configured project status and mapped-item counts. Project-level and shared destinations are provisioned and reused rather than recreated on each run.

## Correctness and operational choices

- Plane owns task identity and state. Notion can generate its own Unique ID, but that is a separate Notion-assigned key; the data source does not enforce uniqueness on the incoming Plane `work_item_id` property. The integration relies on a MongoDB unique index on `(notion_data_source_id, plane_work_item_id)` plus that mapping to find the existing Notion page. A source hash lets repeat syncs skip unchanged writes.

A source-property hash makes reruns cheap: when the normalized Plane-derived properties have not changed, the service records an unchanged result and skips the Notion PATCH. Mongo mappings also let it find the same Notion page on the next run. This is idempotent at the application level

```
boolean unchanged = planeSourceHash.equals(mapping.planeSourc
eHash());
if (unchanged)
    return new UpsertResult(false, false, true, null);
```

- The shared job queue serializes work through one worker, stores run outcomes, and keeps slower Notion provisioning off the config-save response. The UI can show PROVISIONING, UPDATING, READY, or FAILED instead of leaving the user to guess whether work completed.
- The prototype deliberately uses one worker and UTC calendar scheduling. Multi-instance coordination, stuck-job recovery, and timezone/DST handling are production-hardening work, not properties I claim for this demo.

## Live demo path (about 10 minutes)

1. Start with the synthetic Plane workspace and show the selected project and the kind of work a Notion-based project lead wants to see.

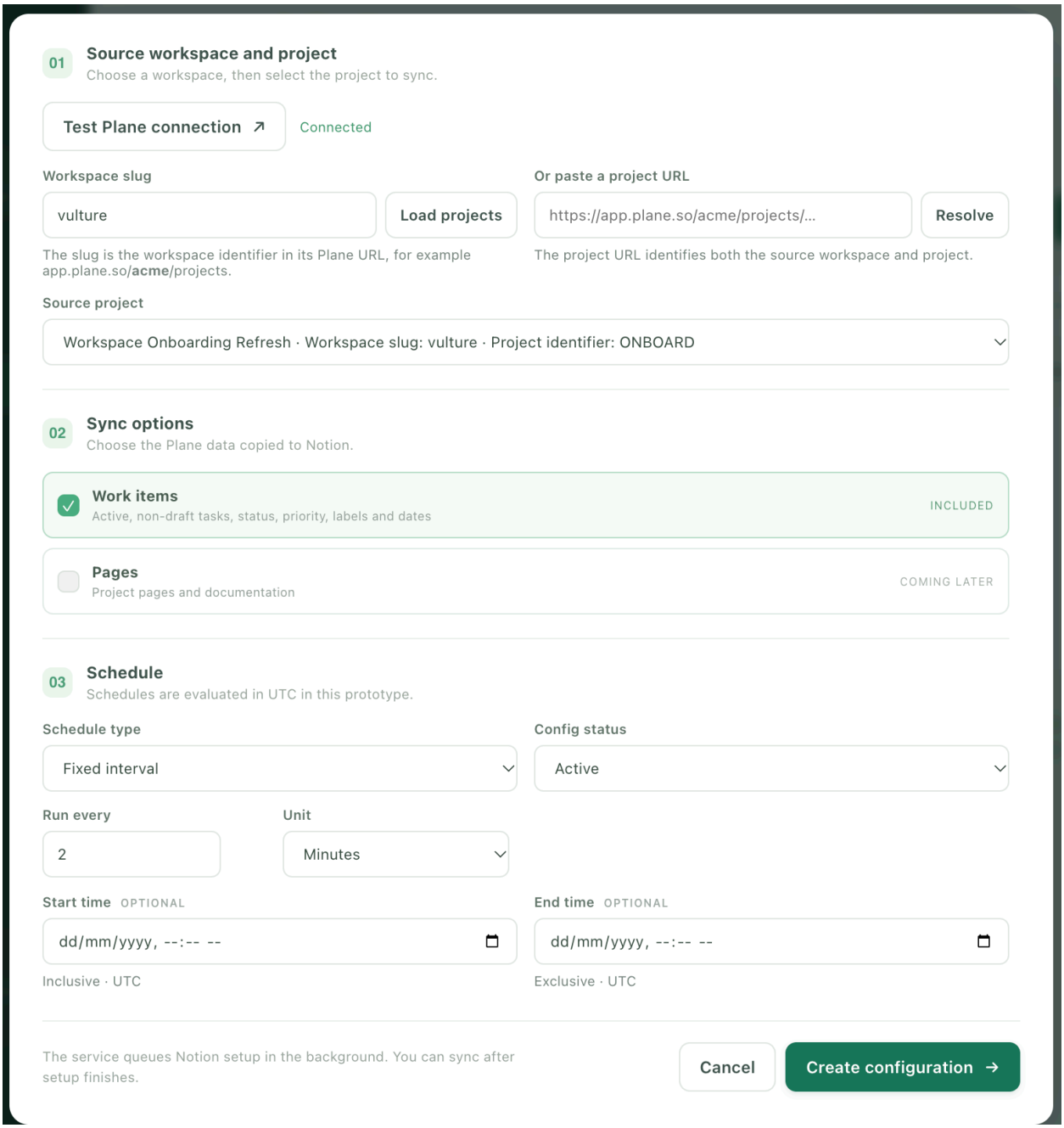
2. In the local dashboard, open the project configuration and its Notion destination. Explain the parent → workspace → project → database hierarchy, then show how saving a configuration queues provisioning and moves through its visible operation state.
3. Run a project sync. Open the resulting Notion project database and show the mapped state, priority, assignee/labels, due date, and source link on a task page.
4. Paste a task URL from that configured project and run it again. Show the run history reporting the item as unchanged, with no duplicate Notion page.
5. Paste a task URL from an unconfigured project. Show that it is routed to the shared ad hoc database and that the row still identifies its workspace and project.
6. Finish in the dashboard's run history: connect the visible job outcome to the Notion rows and point out that Plane remains the place to edit task state.

## Observed prototype walkthrough: ONBOARD (4 Nov 2026)

These screenshots capture the local prototype flow shared during the ONBOARD demo on 4 November 2026. They show asynchronous configuration, the first project sync, a task change flowing into Notion, and a repeat sync that leaves the task list duplicate-free.

### 1. Configure the source project

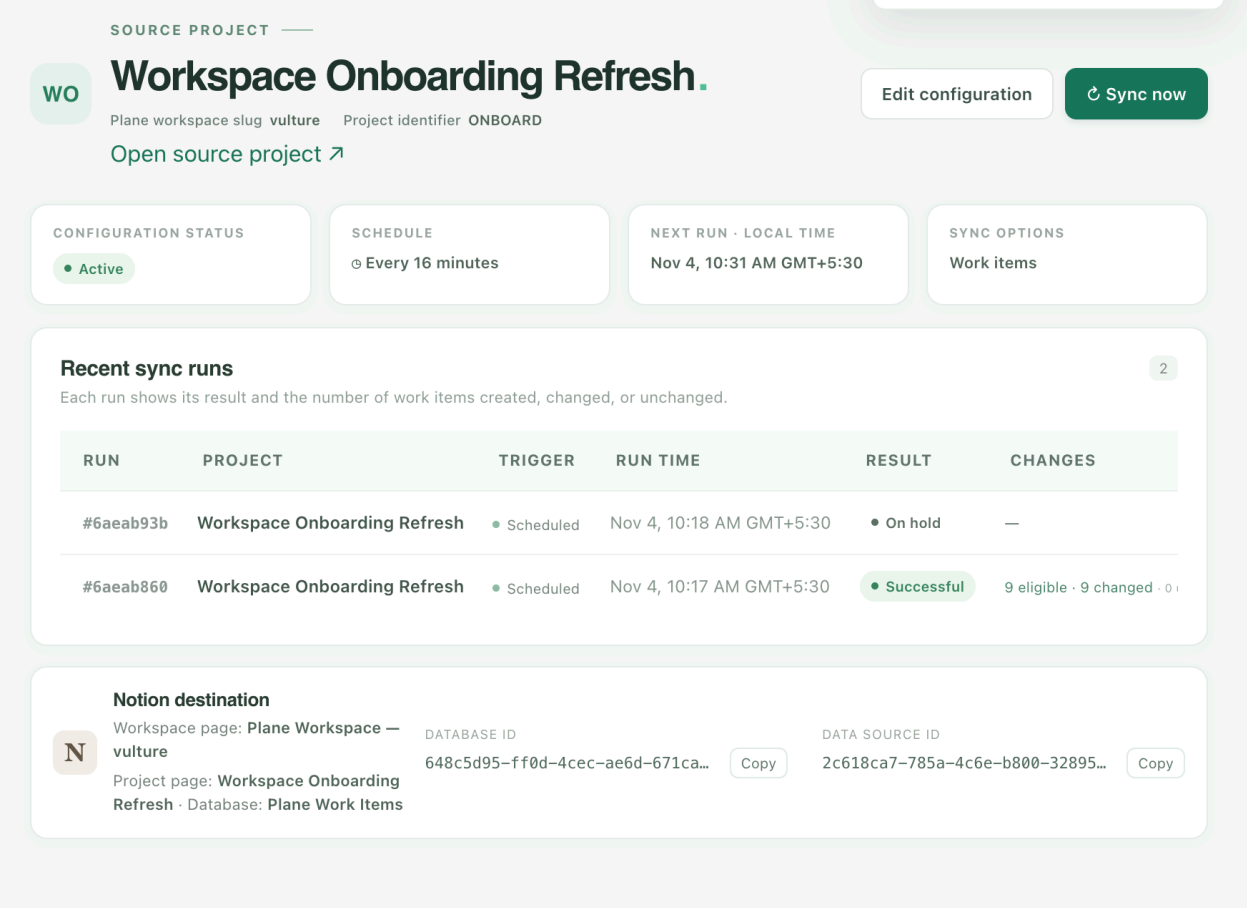
The configuration targets the vulture workspace and ONBOARD project, with work items enabled and pages deferred. The API records the request and the worker provisions the configuration asynchronously; the timeline shows the job moving from PENDING to SUCCESS, then the configuration becoming ACTIVE / READY.




Configuration form for the ONBOARD project.

## 2. Keep configuration updates safe

While changing the schedule from a two-minute to a sixteen-minute interval, the pre-existing scheduled task was held. The update completed successfully, and the run view shows the earlier successful nine-item run alongside the updated schedule.




Configuration view showing the held run and updated schedule.

### 3. Inspect the first Notion project view

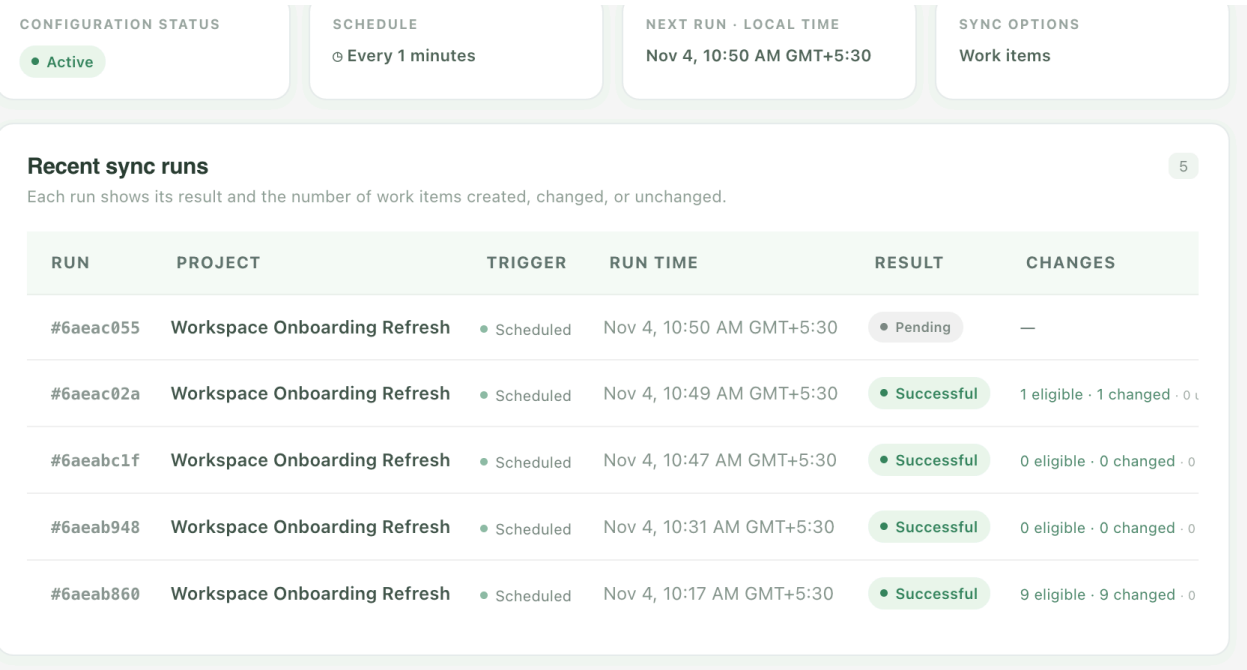
After the initial full sync, the Notion view contains nine ONBOARD work items with their Plane identifiers, statuses, and labels. This establishes the useful project view before demonstrating an incremental change.



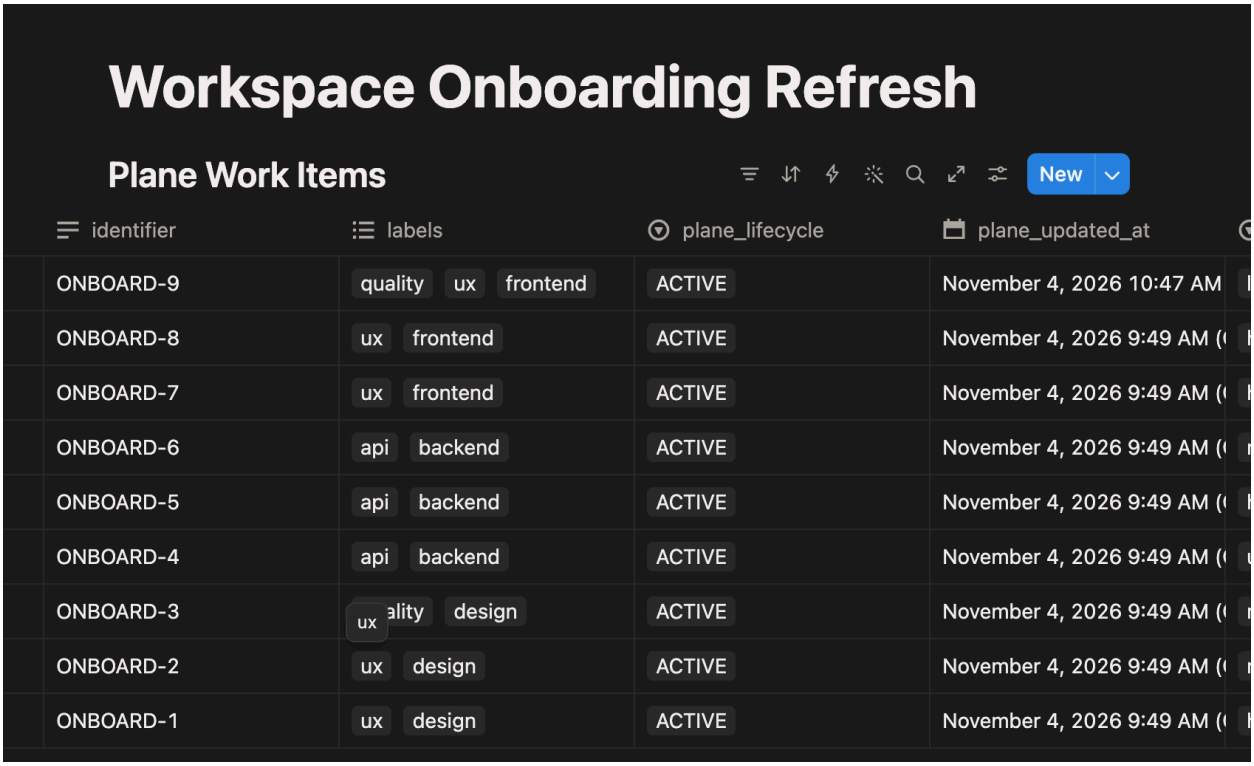

Initial Notion view after the first ONBOARD sync.

### 4. Change a task in Plane and verify the result

ONBOARD-9 was updated in Plane to add the ux label. The resulting one-item run reports 1 eligible and 1 changed item, and the Notion row shows ux alongside its existing quality and frontend labels.



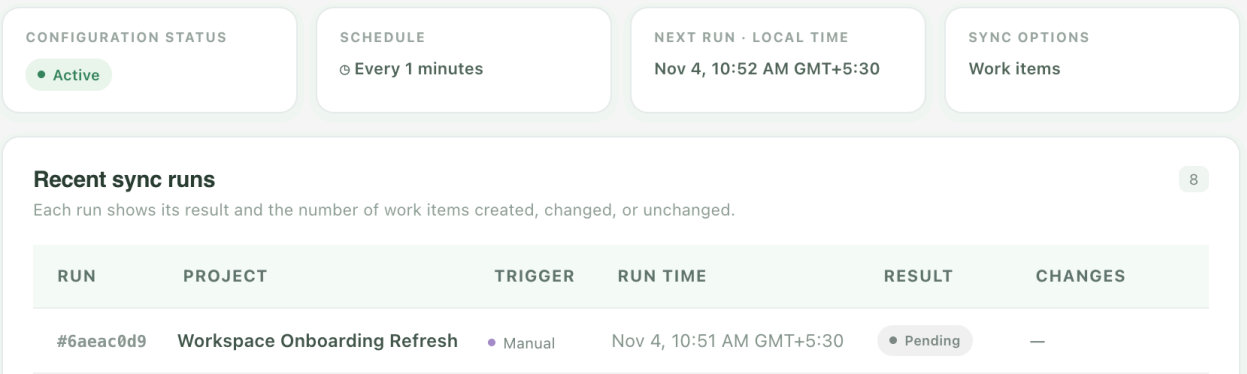

Successful one-item sync: 1 eligible, 1 changed.



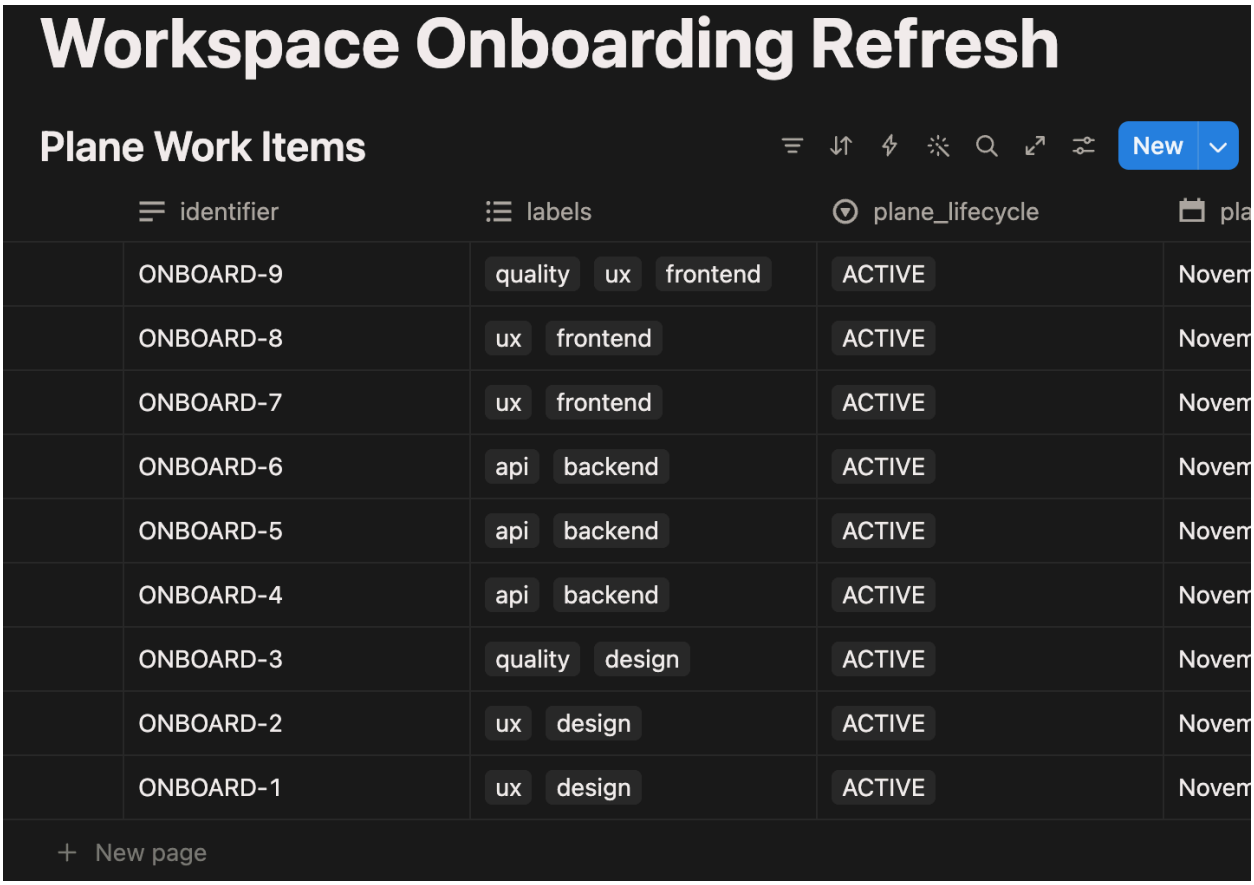

Notion reflects the added ux label on ONBOARD-9.

## 5. Repeat the task sync

A manual sync was queued for ONBOARD-9. The final Notion view still contains ONBOARD-1 through ONBOARD-9 once each, with no tenth row created by the replay.




Manual task sync queued for ONBOARD-9.




Final Notion view: nine ONBOARD rows, with no duplicate.

This is a captured local-prototype walkthrough, not a throughput benchmark. It demonstrates asynchronous configuration handling, safe scheduling behavior during updates, a changed task reflected in Notion, and a repeat sync leaving the visible task set duplicate-free.

In the latest synthetic end-to-end run, one configured project sync completed with 3 eligible / 3 created / 0 failed. A configured task-URL run reported 1 eligible / 1 unchanged. Two task URLs from unconfigured projects each created one row in the shared ad hoc database. The UI showed queued work complete in roughly 10–12 seconds. These are prototype checks with synthetic data, not production service-level claims.

### UI fixes that make async work understandable

I fixed two user-visible failure modes discovered while using the dashboard: after enqueueing, the current run list must show the new job; and when filters or configs change quickly, a slower old response must not overwrite the newer selection. The UI also exposes an explicit force option when manually syncing an archived config, matching the backend rule, and shows PROVISIONING / UPDATING rather than implying Notion setup finished when the request was only accepted. I also clarified workspace versus project labels, source/destination terms, schedules and statuses, and separated the dashboard's recent runs from filterable, paginated history.

Trimmed excerpt from App.tsx:

```
const requestVersion = ++runRequestVersion.current;
const result = await api.listJobs(filters);
if (requestVersion !== runRequestVersion.current) return;
setJobs(result.items);

// after a queued mutation, refresh the view the user is look
ing at
await refreshCurrentView();
```

## What I would harden next

- The current incremental path reads active, non-draft items. Plane archive/unarchive changes can leave updated_at unchanged, so a checkpoint alone cannot reliably detect every reactivation or item moved out of scope. I would add reconciliation or a trustworthy event feed before promising complete lifecycle parity.
- For a production deployment, I would add safe multi-worker coordination, stuck-run recovery and retention policies, and customer/workspace-specific connection management. I would decide on webhooks only after confirming the network can securely receive them.
- I would keep the one-way contract until there is a clear write-back use case and a defined conflict-resolution model. Notion should stay a useful collaboration view, not a competing task authority.

## How I used AI across the project

I used AI coding agents primarily for execution and verification. I designed the architecture and code organization: the shared queue/worker model, data-model ownership, repository contract, Plane and Notion client boundaries, service/controller split, and UI operation flow. I set the scope and acceptance criteria. Agents helped implement and refactor code, maintain tests and documentation, run E2E flows, and iterate on UI; I reviewed and manually verified the results.

### 1. Define scope before implementation

I used AI to explore possible problems (gave an initial set of possible options), audiences, and constraints, then made the final scope choice: a custom, one-way Plane → Notion projection with Plane authoritative. I defined the first iteration around active work items, project/task sync, and run visibility, and deferred two-way write-back, comments, attachments, custom field mapping, and broader lifecycle reconciliation.

### 2. Build bottom-up, one layer at a time

I chose the bottom-up sequence: models → repositories → provider clients → services → controllers → frontend. I set the contracts and architecture at each

layer, then directed the agent to implement/refactor that layer and its callers. I reviewed the result before moving upward.

- Data-model review: I personally examined the data beans field by field and decided where each responsibility belongs. Project records own Plane metadata and configured Notion destination IDs; configs own sync options, schedules, and operation state; mappings own the source-item-to-Notion-page link and source hash; shared ad hoc destination IDs live separately. I made schedule/options strongly typed, and reviewed indexes against actual access patterns, removing or avoiding indexes without a clear query purpose.
- Repository design: I inventoried the repositories and the operations the services actually need. I chose one application-specific interface backed by one MongoTemplate implementation per repository, replacing an earlier split with two interfaces and one implementation. I introduced MongoQueryOptions for reusable filters, sorting, and pagination. The agent carried out the refactor and updated callers/tests under that design.
- Provider boundaries: I defined the Plane and Notion client responsibilities. PlaneClient owns Plane API and URL behavior; NotionClient owns Notion pages, data sources, and work items; NotionBlockClient owns block and summary operations. The agent moved code to those boundaries under my direction.
    - The next refactor would standardize provider-client contracts, separating transport, API error classification, retries, and resource-specific operations.
- Services and controllers: I kept business workflows in services and HTTP request/response handling in controllers. I set the boundary; the agent implemented the corresponding refactors after the model, repository, and client contracts were settled.

## 3. Keep code hygiene and tests in the loop

I set the coding hygiene bar: clear package roles across common, models, repositories, services, and controllers; enums for fixed state sets instead of repeated magic strings; JavaDocs; an AI context file; and repository coding

guidelines. I directed the agent to apply those conventions and keep unit tests in place as the layers changed.

### 4. Use end-to-end evidence to change the design

I directed an agent to run end-to-end flows, then inspected the UI and behavior myself. Those runs exposed latency in config creation and updates because Notion provisioning and schedule work were coupled to the request. I made the architectural decision to move that work to the durable queue and worker, with explicit operation states in the UI; the agent implemented the change and reran the happy path until configuration, sync, and Notion writes completed.

I directed agentic UI iterations, then manually reviewed the rendered screens, labels, status, layout, accessibility, and click-to-result behavior. That review led to fixes such as refreshing the visible run history after enqueue and ignoring stale responses when filters or configs changed.

## Other integration ideas I considered

I compared the options by user value, source-of-truth clarity, integration depth, data sensitivity, and whether I could demonstrate the full path safely.

- Airflow → Notion: a useful DAG/run-status view; pause or trigger controls would need stronger authorization and audit handling.
- VictoriaLogs → Notion: incident summaries and trends could help teams share context, but raw log streams are too large and noisy for Notion.
- Insomnia → Notion: Similar to the existing Postman marketplace connector.
- Substack → Notion: A straightforward personal reading database, but it is less connected to a business workflow.
- Expense tracking → Notion: reconciliation is interesting, but multiple sources and sensitive financial data make the first version broader and higher-risk.

I chose Plane because it gave me the clearest end-to-end engineering story: resolve source records, preserve source identity, sync incrementally and idempotently, and present useful work context in Notion while keeping Plane authoritative. Another reason was ecosystem coverage: Notion already supports

Jira and Asana integrations, so I saw a Plane connector as a natural way to broaden the task-management tools teams can bring into Notion.

## 30-minute run of show

- 0–5 min: problem, audience, business value, other ideas, and why Plane.
- 5–9 min: scope priorities, architecture, Notion model, and source-of-truth boundary.
- 9–18 min: live configuration, sync, rerun, and Notion inspection.
- 18–24 min: code excerpts, correctness, async provisioning, and bulk evidence.
- 24–30 min: AI workflow, production limits, roadmap, and questions.

## Closing

This project shows how I take an operational need from discovery through a working integration: define the ownership boundary, resolve and map source data, deliver it in a Notion structure people can use, and make the sync observable and safe to rerun. The next step is to harden lifecycle reconciliation and runtime operations for the customer’s actual network and scale.